Performance measurement and profiling

1024 x 1024 x 1024 GEMM, FP16, Iluvatar BI-V150

Order	Demonstration	Question
00	Environment and smoke test	Can GPU work complete?
01	Offline and Server benchmarks	What does each timing measure?
02	Roofline	What does the model predict?
03	Profiling	What evidence explains the gap?
04	Summary and output checks	Are the results consistent?

Historical reference: 15 Oct snapshot and six process runs. Reset-environment verification: page 17.

Check the environment

```
git clone https://gitee.com/sunxt-0719/lab5-profiler.git
cd lab5-profiler
source env_corex.sh
bash setup.sh
python3 00_check_env.py
python3 00_smoke_cuda.py
```

Check	Recorded result
Device	Iluvatar BI-V150, logical cuda:0
Software	Python 3.12.11 | PyTorch 2.10.0 | Triton 3.6.0
CoreX wheel	2.10.0+corex.4.5.0
Smoke test	[smoke] OK

Keep the platform PyTorch/Triton packages. Source: results/env_check.json and validated smoke workflow.

Run Offline with the completed defaults

```
python3 01_benchmark.py --scenario offline \
  --M 1024 --N 1024 --K 1024 --repeats 3
```

torch + baseline + autotune are already enabled. No configuration edit is required.

```
[check] baseline: {'status': 'PASS', ...}
[check] autotune: {'status': 'PASS', ...}
[done] wrote .../results/benchmark_offline.json
```

Check three metric rows and 48 samples per implementation: 16 queries x 3 rounds.

PyTorch does not print a Triton validation PASS. JIT compilation, validation and autotuning precede measured rounds.

Read the original Offline results

Implementation	Service median, ms	Service p99, ms	Samples/s	Effective TFLOP/s
torch	0.0749	0.0838	11519.5	28.69
baseline	0.3326	0.3453	2946.9	6.46
autotune	0.2224	0.2344	4271.3	9.66

Autotune is 1.50x faster than baseline in this run; its service time is 2.97x PyTorch.

Median: the middle of the samples. p99: an estimate near the slow end of the distribution.

Original snapshot only; 48 samples each. Source: results/benchmark_offline.json > comparison.

Understand the timing and TFLOP/s

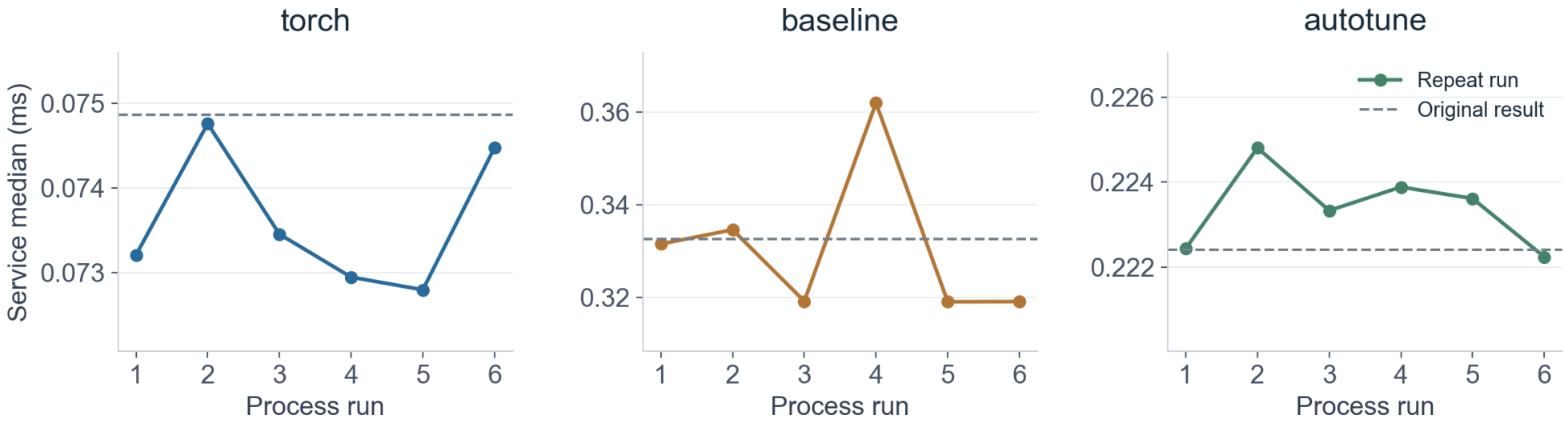
Metric	Timing or arithmetic basis
Service latency	Host dispatch to synchronized completion
Samples/s	Completed queries / measured wall time
Effective TFLOP/s	Useful GEMM FLOPs / median service seconds / 10^12

```
FLOPs = 2 * M * N * K = 2,147,483,648
Effective TFLOP/s = FLOPs / (median_ms * 1e-3) / 1e12
```

Host dispatch and synchronization are included; this is not pure kernel time.

The timing denominator matters when comparing against a device-only peak or a profiler kernel duration.

Check Offline repeatability

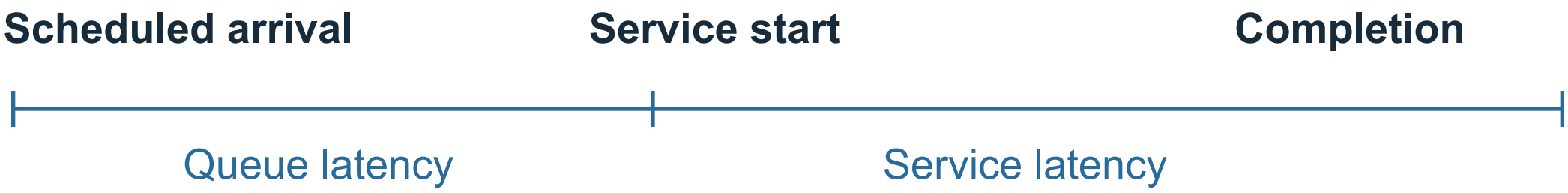


Implementation	Median of run medians, ms	Run-median range, ms	CV of run medians
torch	0.0733	0.0728 - 0.0748	1.11%
baseline	0.3254	0.3191 - 0.3619	5.03%
autotune	0.2235	0.2222 - 0.2248	0.43%

6 runs x 48 = 288 samples/implementation. Same ranking in all 6. Own y scales; dashed line = original.

Run Server and separate the latencies

```
python3 01_benchmark.py --scenario server --repeats 3
```



Response latency = queue latency + service latency

Poisson arrivals, target 8 queries/s, one synchronized worker.
32 queries x 3 rounds = 96 samples per implementation.

Equality holds per request. Percentiles of separate queue and service distributions do not generally add.

Read the original Server results

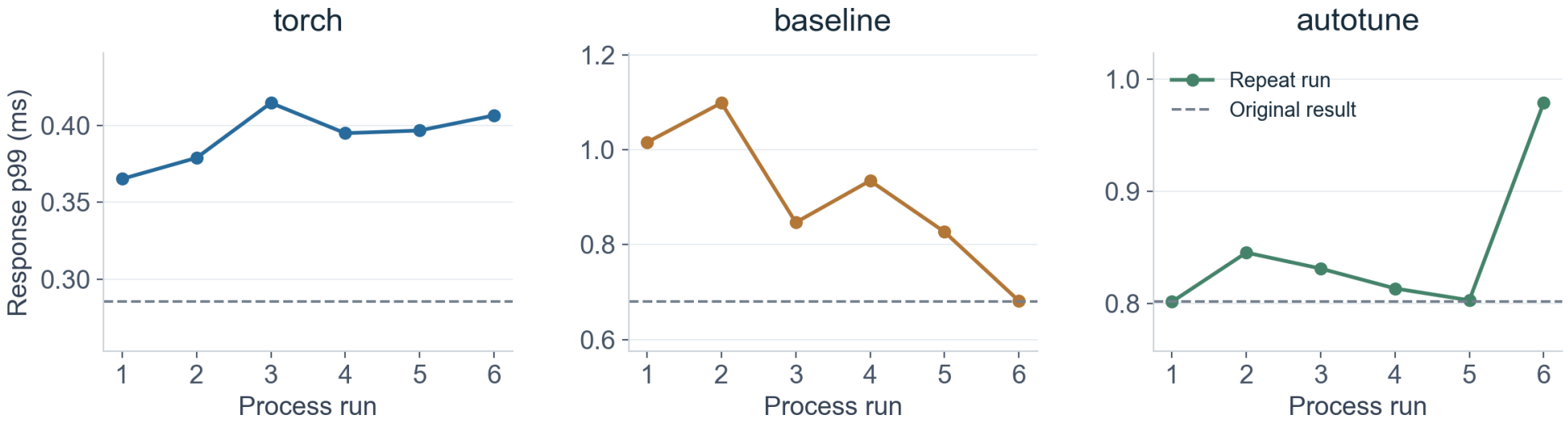
Implementation	Service median, ms	Queue median, ms	Response p99, ms	Samples/s
torch	0.1250	0.0994	0.2854	9.020
baseline	0.4945	0.1008	0.6795	9.019
autotune	0.4380	0.1019	0.8018	9.019

At low offered load, completed throughput follows the arrival schedule.

Server SLA checks response p99 against 500 ms. All three original results passed.

Original snapshot; 96 samples each. Approximately 9 queries/s here is not a GPU capacity measurement.

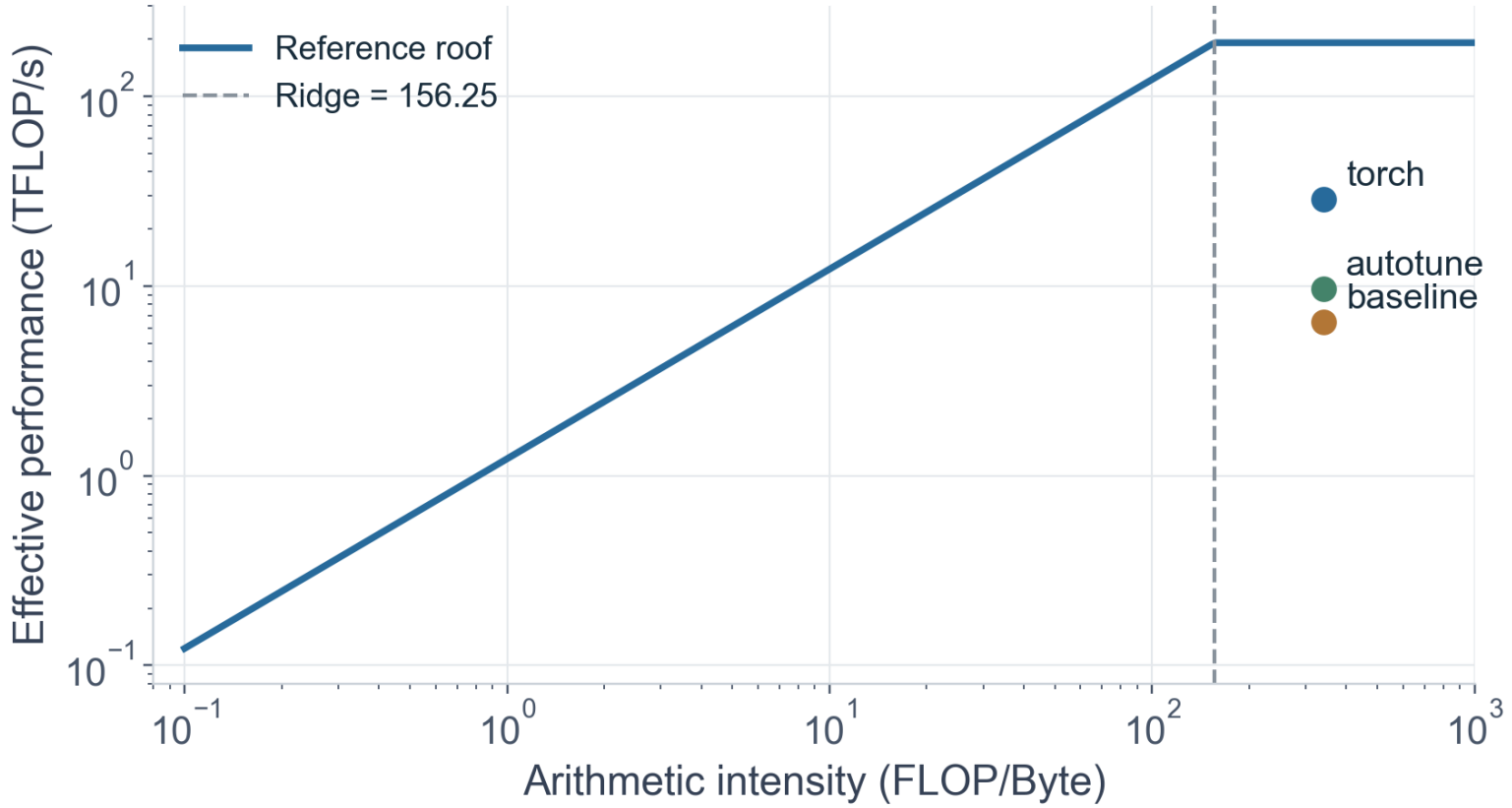
Server tail latency varies across runs



Implementation	Service-median CV	Response-p99 range, ms	Response-p99 CV
torch	2.72%	0.365 - 0.415	4.61%
baseline	1.19%	0.682 - 1.099	16.47%
autotune	1.44%	0.802 - 0.979	7.98%

Six new processes, 96 samples each (576 per implementation). Same arrival seed 42; this is not a tail-SLA certification.

Build the reference Roofline




```
python3 02_roofline.py
```

Reference inputs
192 TFLOP/s
1228.8 GB/s

Ridge: **156.25** FLOP/Byte
GEMM intensity: **341.33** FLOP/Byte

Peak parameters are unverified reference assumptions. Points use original Offline service timing, including host overhead.

Interpret the model without overclaiming

```
FLOPs = 2*M*N*K
Bytes ~= (M*K + K*N + M*N) * 2
I = FLOPs / Bytes = 341.33 FLOP/Byte
I_ridge = 192e12 / 1228.8e9 = 156.25 FLOP/Byte
```

Same problem and dtype -> same modeled intensity.

compute-side describes a model region.
actual bottleneck: undetermined

Real cache traffic, redundant transfers and on-chip access costs can differ between implementations.

The simplified traffic model counts A, B and C once. Use profiling and controlled experiments to investigate the actual gap.

Capture the selected autotune configuration

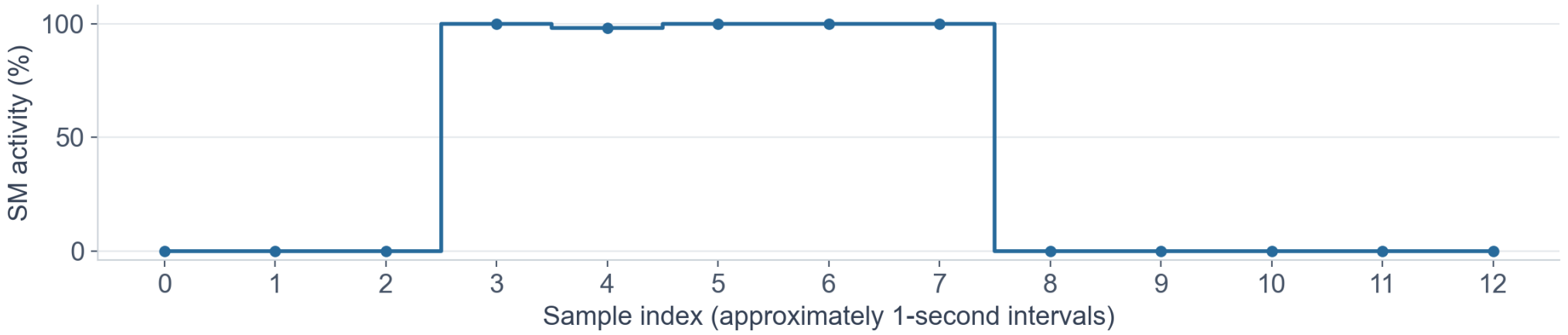
```
python3 03_profiling.py --capture
python3 03_profiling.py --impl autotune --capture \
  --out results/autotune/diagnosis.json
```

```
[CAPTURED] ixsmi
[CAPTURED] ixsys
[CAPTURED] ixkn-cli
[model] autotune: compute-side; actual bottleneck undetermined
```

Replay the recorded winner: 128 x 64 x 32, 4 warps, 2 stages.
No new autotune search runs under the profiler.

Default capture: baseline. Optional --out keeps autotune evidence separate. Look for three CAPTURED messages per run.

Inspect activity and trace evidence



Tool	Original capture	Interpretation
ixsmi	13 samples; maximum SM activity 100%	Device was busy during the sampled interval
ixsys	24 _matmul_kernel records	Trace contains GEMM activity

Source: results/profiling/ixsmi_dmon.log and profiling_summary.json. GPU activity is not percent of peak FLOP/s.

Read the kernel report

Metric	Original capture	What to read from it
Kernel duration	0.1277 ms	One profiled GPU kernel
Achieved / theoretical occupancy	24.55% / 25.00%	Active warps vs reported maximum
TCU efficiency	17.91%	Vendor tensor-compute indicator
LSU utilization	86.25%	Load/store-unit activity
L2 cache load hit rate	95.01%	Hits among the counted L2 loads
Shared store bank conflicts	1,859,584	Shared-memory access contention

Single original autotune capture: results/profiling/ixkn_details.log. Kernel timing and service timing have different scopes.

Interpret register and memory indicators

Observation	Supported interpretation
92 vector registers/thread	Report identifies VRF as a theoretical occupancy limiter
24.55% achieved occupancy	Close to this launch configuration's 25% theoretical level
Shared store bank-conflict warning	Inspect shared-memory access patterns

Tile, warp count, stages and access layout can affect these indicators.
A follow-up needs validated output and timings without a profiler.

The current script leaves actual bottleneck undetermined.

Counter warnings generate hypotheses. Higher occupancy or fewer conflicts do not automatically guarantee a speedup.

Validate the complete result set

```
python3 04_summary.py
```

```
[acceptance] PASS
[warnings] offline: 48 samples; p99 is exploratory
[warnings] server: 96 samples; p99 is exploratory
[warnings] Roofline uses unverified reference peak assumptions
```

Checks: required implementations, numerical validation, workload matching, provenance, SLA results and captured artifacts.

Summary: service median / response p99.
Offline response includes waiting behind the batch.

The excerpt collects the original PASS and warnings; their printed order may differ. PASS is not bottleneck certification.

Check the final outputs

Implementation	Reset run: Offline service median, ms	Reset run: Server service median, ms
torch	0.0748	0.1253
baseline	0.3197	0.4866
autotune	0.2236	0.4379

16 Oct reset run: summary PASS; 18 regression tests passed.
Baseline + autotune: ixsmi, ixsys and ixkn captured.

Finish with JSON results, Roofline PNG and profiler artifacts.
Autotune improves baseline; PyTorch remains faster.

Fresh clone ca2d97b; 36 tracked files unchanged. Service medians shown here; small-sample and peak warnings remain.

Reference A: all six service medians

Run	Off torch	Off baseline	Off autotune	Srv torch	Srv baseline	Srv autotune
1	0.0732	0.3316	0.2224	0.1258	0.4989	0.4357
2	0.0748	0.3346	0.2248	0.1334	0.5107	0.4476
3	0.0735	0.3193	0.2233	0.1257	0.4947	0.4326
4	0.0729	0.3619	0.2239	0.1317	0.4982	0.4463
5	0.0728	0.3191	0.2236	0.1253	0.4991	0.4447
6	0.0745	0.3192	0.2222	0.1268	0.4942	0.4463

Units: milliseconds. These are process-level service medians, not individual request samples.

Full machine-readable measurements: day2-lab5-repeatability.csv and .json. No repeat was discarded.

Reference B: tail range and selected tiles

Implementation	Server response-p99 minimum, ms	Maximum, ms	CV
torch	0.3653	0.4147	4.61%
baseline	0.6817	1.0989	16.47%
autotune	0.8018	0.9791	7.98%

Scenario	Autotune tile: selection count	Warps / stages
offline	128x64x32: 6/6	4 / 2
server	64x64x32: 6/6	4 / 2

Balanced implementation order; fixed arrival seed 42.
Warm compiler cache; no profiler during repeated timing.

CV = sample standard deviation / mean x 100%, calculated across six process-level estimates.

Reference C: provenance and file guide

Source	Used for
results/benchmark_offline.json	Original Offline table and Roofline points
results/benchmark_server.json	Original Server table and SLA
results/roofline.json	Modeled intensity, ridge and peak provenance
results/profiling/ixkn_details.log	Single original autotune kernel report
results/diagnosis.json + profiling_summary.json	Capture status, configuration and artifact hashes
results/summary.json + env_check.json	Original acceptance result and environment

Historical paths were under /root/day2-lab5; current checkout: /root/lab5-profiler.
Repeat study: 2026-10-15 18:24 to 18:28 Asia/Shanghai.
Source checkout: da32c64, with the existing local configuration preserved.

Original Offline ID: dae1081c4fab49d3aa68a7ef0210d1ec
Original Server ID: 67c3c54354324b23b96d6e9bdb64b302

Historical evidence: day2-lab5-reference-data.zip. Reset run: lab5-profiler-reset-run-20261016.tar.gz (ca2d97b).